

ECE3403 - Data Analytics and Optimization using Python

Dr. Ahmed Farouk
Spring - 2025

Lecture 2

Data Preprocessing – Handling Missing Values and Outliers

Python Libraries

- Pandas libraries
- NumPy libraries

Pandas libraries

- Pandas is a powerful Python library used for data manipulation and analysis.
- It provides data structures like DataFrame (a 2D labeled table) and Series (a 1D labeled array) to handle structured data efficiently.
- Pandas offers functions for data cleaning, transformation, aggregation, and visualization, making it essential for tasks like reading/writing CSV or Excel files, filtering data, handling missing values, and performing statistical operations.
- It is widely used in data science, machine learning, and analytics.

NumPy libraries

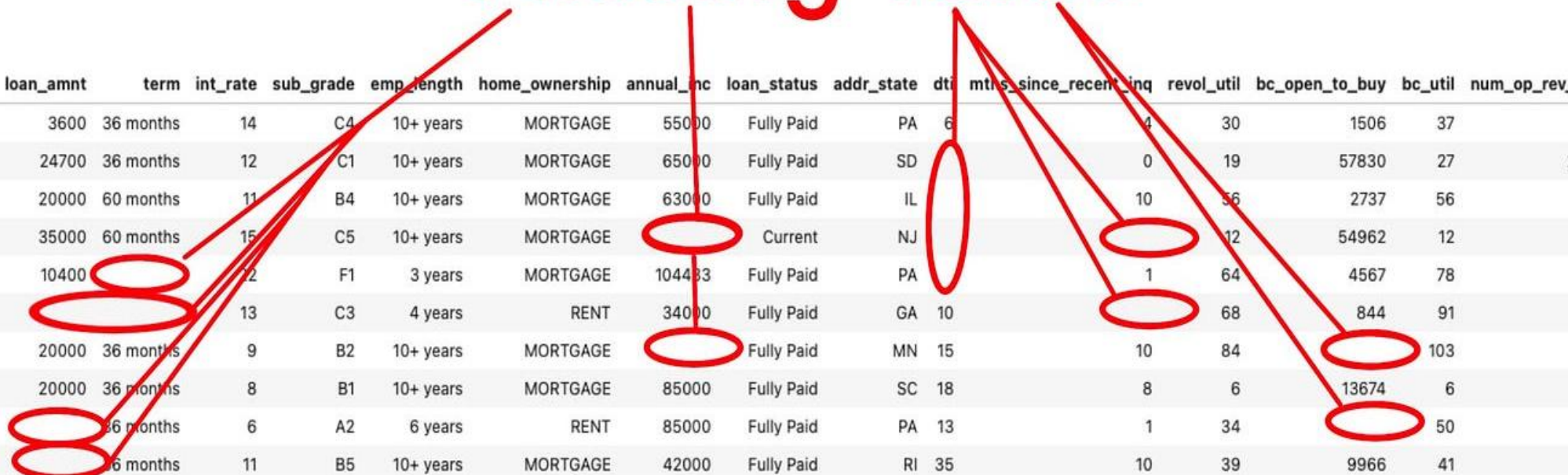
- NumPy (Numerical Python) is a fundamental Python library for numerical computing.
- It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on them efficiently.
- NumPy is widely used in data science, machine learning, and scientific computing because of its speed and ability to handle large datasets.
- It also serves as the foundation for other libraries like Pandas, SciPy, and TensorFlow.

Handling Missing Values

What is Missing Values?

- Missing data is defined as the values or data that is not stored (or not present) for some variable/s in the given dataset.

Missing value



	loan_amnt	term	int_rate	sub_grade	emp_length	home_ownership	annual_inc	loan_status	addr_state	dti	mtls_since_recent_inq	revol_util	bc_open_to_buy	bc_util	num_op_rev_tl
0	3600	36 months	14	C4	10+ years	MORTGAGE	55000	Fully Paid	PA	6	4	30	1506	37	4
1	24700	36 months	12	C1	10+ years	MORTGAGE	65000	Fully Paid	SD		0	19	57830	27	20
2	20000	60 months	11	B4	10+ years	MORTGAGE	63000	Fully Paid	IL		10	56	2737	56	4
3	35000	60 months	15	C5	10+ years	MORTGAGE		Current	NJ			12	54962	12	10
4	10400		12	F1	3 years	MORTGAGE	104433	Fully Paid	PA		1	64	4567	78	7
5			13	C3	4 years	RENT	34000	Fully Paid	GA	10		68	844	91	4
6	20000	36 months	9	B2	10+ years	MORTGAGE		Fully Paid	MN	15	10	84		103	9
7	20000	36 months	8	B1	10+ years	MORTGAGE	85000	Fully Paid	SC	18	8	6	13674	6	3
8		36 months	6	A2	6 years	RENT	85000	Fully Paid	PA	13	1	34		50	13
9		36 months	11	B5	10+ years	MORTGAGE	42000	Fully Paid	RI	35	10	39	9966	41	5

How is a Missing Value Represented in a Dataset?

- Missing values in a dataset can be represented in various ways, depending on the source of the data and the conventions used
 - **NaN (Not a Number):** In many programming languages and data analysis tools, missing values are represented as NaN. This is the default for libraries like Pandas in Python.
 - **NULL or None:** In databases and some programming languages, missing values are often represented as NULL or None. For instance, in SQL databases, a missing value is typically recorded as NULL.
 - **Empty Strings:** Sometimes, missing values are denoted by empty strings (""). This is common in text-based data or CSV files where a field might be left blank.
 - **Special Indicators:** Datasets might use specific indicators like -999, 9999, or other unlikely values to signify missing data. This is often seen in older datasets or specific industries where such conventions were established.
 - **Blanks or Spaces:** In some cases, particularly in fixed-width text files, missing values might be represented by spaces or blank fields.

Reasons for Missing Data

- **Human Errors:** data entry mistakes (e.g., skipping fields in surveys/forms).
- **Survey & Questionnaire Issues:** respondents skip questions they find too personal or irrelevant.
- **System & Technical Errors:** database failures or system crashes.
- **Data Privacy & Restrictions:** some sensitive data (e.g., personal identifiers) might be removed due to privacy regulations (GDPR).
- **Filtering & Conditional Logic:** some fields are only relevant based on previous responses (e.g., "If yes, specify reason").

Types of Missing Values

- Understanding the types of missing values in datasets is crucial for effectively handling missing data and ensuring accurate analyses:
- Missing Completely At Random (MCAR)
 - MCAR occurs when the probability of data being missing is uniform across all observations. There is no relationship between the missingness of data and any other observed or unobserved data within the dataset. This type of missing data is purely random and lacks any discernible pattern.
- Example: In a survey about library books, some overdue book values in the dataset are missing due to human error in recording.

Types of Missing Values

- **Missing At Random (MAR)**

- MAR data occurs when the probability of data being missing depends only on the observed data and not on the missing data itself. In other words, the missingness can be explained by variables for which you have complete information. There is a pattern in the missing values, but this pattern can be explained by other observed variables.
- Example: In a survey, 'Age' values might be missing for those who did not disclose their 'Gender'. Here, the missingness of 'Age' depends on 'Gender', but the missing 'Age' values are random among those who did not disclose their 'Gender'.

- **Missing Not At Random (MNAR)**

- MNAR occurs when the missingness of data is related to the unobserved data itself, which is not included in the dataset. This type of missing data has a specific pattern that cannot be explained by observed variables.
- Example: In a survey about library books, people with more overdue books might be less likely to respond to the survey. Thus, the number of overdue books is missing and depends on the number of books overdue.

Handling Missing Data

- Handling Missing Data
 - **Deletion:** This involves removing rows or columns with missing values. This is a straightforward method, but it can be problematic if a significant portion of your data is missing. Discarding too much data can affect the reliability of your conclusions.
 - **Imputation:** This replaces missing values with estimates. There are various imputation techniques, each with its strengths and weaknesses. Here are some common ones:
 - **Mean/Median/Mode Imputation:** Replace missing entries with the average (mean), middle value (median), or most frequent value (mode) of the corresponding column. This is a quick and easy approach, but it can introduce bias if the missing data is not randomly distributed.
 - **K-Nearest Neighbors (KNN Imputation):** This method finds the closest data points (neighbors) based on available features and uses their values to estimate the missing value. KNN is useful when you have a lot of data and the missing values are scattered.
 - **Model-based Imputation:** This involves creating a statistical model to predict the missing values based on other features in the data. This can be a powerful technique, but it requires more expertise and can be computationally expensive.`pen_spark`.

Checking for Missing Values in Python

- The first step in handling missing values is to carefully look at the complete data and find all the missing values.
- The following code shows the total number of missing values in each column.

```
import pandas as pd
train_df = pd.read_csv("train.csv")
#Find the missing values from each column
print(train_df.isnull().sum())
```

- It also shows the total number of missing values in the entire data set.
- #Find the total number of missing values from the entire dataset
- `train_df.isnull().sum().sum()`
- #Output
- 149

Checking for Missing Values in Python

```
import pandas as pd
# Read the CSV file into a DataFrame
file_path = 'your_dataset.csv' # Replace with your CSV file path
df = pd.read_csv(file_path)
# Get the number of missing values in each column
missing_values = df.isnull().sum()
# Filter columns with missing values
columns_with_missing_values = missing_values[missing_values > 0]
# Print the column name and the number of missing values
if not columns_with_missing_values.empty:
    print("Columns with missing values:")
    for column, num_missing in columns_with_missing_values.items():
        print(f"{column}: {num_missing} missing values")
else:
    print("No columns have missing values.")
```

Deletion

- This approach is not recommended. However It is one of the quick techniques one can use to deal with missing values.
- If the missing value is of the type Missing Not At Random (MNAR), then it should not be deleted.
- Listwise Deletion (Complete Case Analysis) Removes entire rows where at least one column has a missing value.
 - Best used when missing data is random and does not cause data loss.
- Column-wise Deletion (Dropping Columns) Removes entire columns that contain missing values.
 - Useful when a column has too many missing values to be useful.
- The disadvantage of this method is one might end up deleting some useful data from the dataset.

Deletion

```
import pandas as pd
# Sample dataset with missing values
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', None],
    'Age': [25, None, 30, 22, 28],
    'Salary': [50000, 60000, None, 45000, 52000]
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
# 1. Listwise deletion (drop rows with any missing values)
df_listwise = df.dropna()
print("\nAfter Listwise Deletion:")
print(df_listwise)
# 2. Column-wise deletion (drop columns with missing values)
df_columnwise = df.dropna(axis=1)
print("\nAfter Column-wise Deletion:")
print(df_columnwise)
```

Original DataFrame:

	Name	Age	Salary
0	Alice	25.0	50000.0
1	Bob	NaN	60000.0
2	Charlie	30.0	NaN
3	David	22.0	45000.0
4	None	28.0	52000.0

After Listwise Deletion:

	Name	Age	Salary
0	Alice	25.0	50000.0
3	David	22.0	45000.0

After Column-wise Deletion:

Empty DataFrame
Columns: []
Index: [0, 1, 2, 3, 4]

Mean_(Imputing the Missing Value)

$$m = \frac{\text{sum of the terms}}{\text{number of terms}}$$

Why Use Mean Imputation?

- Preserves Data Distribution: Mean imputation maintains the overall trend in the dataset.
- Simple and Effective: It is easy to implement and works well for numerical data.
- Prevents Data Loss: Unlike row deletion, it retains useful information.
- Useful for Machine Learning: Many ML models do not handle missing data directly, so imputation is required.

Mean

```
# Step 1: Import Required Libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
# Step 2: Create a Sample Dataset with Missing Values
```

```
# Creating a dataset with missing values
```

```
data = {
```

```
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emma'],
```

```
    'Age': [25, 30, np.nan, 40, 35], # Missing value in 'Age'
```

```
    'Salary': [50000, 54000, 52000, np.nan, 58000] # Missing value in 'Salary'
```

```
}
```

```
# Convert dictionary to a DataFrame
```

```
df = pd.DataFrame(data)
```

```
# Display the dataset
```

```
print("Original Dataset:")
```

```
print(df)
```

Mean

- Output:

	Name	Age	Salary
0	Alice	25.0	50000.0
1	Bob	30.0	54000.0
2	Charlie	NaN	52000.0
3	David	40.0	NaN
4	Emma	35.0	58000.0

- We have missing values (NaN) in both the "Age" and "Salary" columns.

Mean

- Step 3: Check for Missing Values
- Before filling missing values, it's good practice to check where NaN values exist.

```
print("Missing values per column:")  
print(df.isnull().sum()) # Counts missing values in each column
```

- Output

```
Missing values per column:  
Name      0  
Age       1  
Salary    1  
dtype: int64
```

- The "**Age**" and "**Salary**" columns each have 1 missing value.

Mean

- Step 4: Fill Missing Values Using Mean
- To replace missing values with the column mean, use `fillna()`.

```
# Fill missing values in 'Age' column with its mean  
df['Age'].fillna(df['Age'].mean(), inplace=True)
```

```
# Fill missing values in 'Salary' column with its mean  
df['Salary'].fillna(df['Salary'].mean(), inplace=True)
```

```
# Display the updated DataFrame  
print("Dataset after Mean Imputation:")  
print(df)
```

Mean

- Output:

	Name	Age	Salary	
0	Alice	25.0	50000.0	
1	Bob	30.0	54000.0	
2	Charlie	32.5	52000.0	# Missing 'Age' replaced with mean (32.5)
3	David	40.0	53500.0	# Missing 'Salary' replaced with mean (53500.0)
4	Emma	35.0	58000.0	

- The missing value in "Age" was replaced with 32.5 (mean of available ages).
- The missing value in "Salary" was replaced with 53500 (mean of available salaries).

Mean

- 5. Alternative: Using `SimpleImputer` from `sklearn`
 - (FYN) If you are working with machine learning, it's better to use `SimpleImputer` from `scikit-learn`.

```
from sklearn.impute import SimpleImputer
```

```
# Creating an imputer object
```

```
imputer = SimpleImputer(strategy='mean')
```

```
# Apply imputer only on numerical columns
```

```
df[['Age', 'Salary']] = imputer.fit_transform(df[['Age', 'Salary']])
```

```
# Display the updated dataset
```

```
print("Dataset after Imputation using SimpleImputer:")
```

```
print(df)
```

Mean

When NOT to Use Mean Imputation

- If the data has extreme outliers, the mean might not be a good representative.
- When Missing Data is Too High: If more than 30% of data is missing, imputation might not be reliable.
- For Categorical Data: Mean imputation works only for numerical data, not categorical data like "Gender" or "City".

Mean Summary

Step	Action
1. Import Libraries	import pandas as pd, import numpy as np
2. Create Dataset	Use pd.DataFrame() with missing values (np.nan)
3. Check Missing Values	df.isnull().sum()
4. Compute Mean & Fill Missing Values	df['column'].fillna(df['column'].mean(), inplace=True)
5. Alternative Using Scikit-Learn	SimpleImputer(strategy='mean')

Median

$$= \begin{cases} X[\frac{n+1}{2}] & \text{if } n \text{ is odd} \\ \frac{X[\frac{n}{2}] + X[\frac{n}{2} + 1]}{2} & \text{if } n \text{ is even} \end{cases}$$

1, 3, 3, **6**, 7, 8, 9

6

1, 2, 3, **4**, **5**, 6, 8, 9

$(4+5)/2 = 4.5$

Why Use Median Instead of Mean?

- Robust to Outliers: Unlike the mean, the median is not affected by extreme values.
- Better for Skewed Data: If a dataset is not normally distributed (e.g., right-skewed or left-skewed), median imputation is preferable.
- Maintains Central Tendency: It ensures that missing values are filled with a representative statistic.

Median

- Example: Handling Missing Data Using Median in Python
- Step 1: Import Required Libraries

```
import numpy as np  
import pandas as pd
```

Median

- Step 2: Create a Sample Dataset

```
data = {  
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],  
    'Age': [25, 30, 35, np.nan, 40, 28, np.nan, 42, 38, np.nan], # Missing values  
    'Salary': [50000, 54000, np.nan, 60000, 65000, 52000, np.nan, 70000, 75000,  
80000] # Missing values  
}  
  
df = pd.DataFrame(data)  
print("Original DataFrame:\n", df)
```

Median

	ID	Age	Salary
0	1	25.0	50000.0
1	2	30.0	54000.0
2	3	35.0	NaN
3	4	NaN	60000.0
4	5	40.0	65000.0
5	6	28.0	52000.0
6	7	NaN	NaN
7	8	42.0	70000.0
8	9	38.0	75000.0
9	10	NaN	80000.0

- There are missing values (NaN) in both the Age and Salary columns.

Median

- Step 3: Compute the Median Values

```
median_age = df['Age'].median()  
median_salary = df['Salary'].median()  
print("Median Age:", median_age)  
print("Median Salary:", median_salary)
```

- Output:

Median Age: 34.0

Median Salary: 62500.0

Median

- Step 4: Replace Missing Values with the Median

```
df['Age'].fillna(median_age, inplace=True)
```

```
df['Salary'].fillna(median_salary, inplace=True)
```

```
print("\nDataFrame after filling missing values:\n", df)
```

Median

- Output:

	ID	Age	Salary
0	1	25.0	50000.0
1	2	30.0	54000.0
2	3	35.0	62500.0
3	4	34.0	60000.0
4	5	40.0	65000.0
5	6	28.0	52000.0
6	7	34.0	62500.0
7	8	42.0	70000.0
8	9	38.0	75000.0
9	10	34.0	80000.0

- Explanation:

- The missing values in the Age column are replaced with 34.0 (the median).
- The missing values in the Salary column are replaced with 62500.0 (the median).

Median

- Alternative Approach: Using SimpleImputer from Scikit-learn
 - If you are working with machine learning models, you can use SimpleImputer from sklearn.impute to handle missing values.

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
df[['Age', 'Salary']] = imputer.fit_transform(df[['Age', 'Salary']])
print("\nDataFrame after using SimpleImputer:\n", df)
```

Median

- When to Use the Median?
- When the data has outliers, as the median is robust to extreme values.
- When the data distribution is skewed.
- When the missing values are random and not a significant proportion of the data.

Mode Imputation

- Mode imputation is a technique used to handle missing data by replacing missing values with the most frequently occurring value (mode) in the column.
- we ensure that the dataset remains complete without introducing much distortion.
- This is particularly **useful for categorical variables**, where using the mean or median wouldn't make sense.
 - Why Use Mode Imputation?
 - It preserves the most common category in a dataset.
 - It maintains categorical integrity without creating biased numerical averages.
 - It is simple and effective for columns with low cardinality.

Mode Imputation

- Let's consider a dataset that contains missing values in a categorical column and apply mode imputation.

- Step 1: Import Necessary Libraries

```
import pandas as pd
import numpy as np
```

- Step 2: Create a Sample Dataset

```
data = {
    'Customer_ID': [101, 102, 103, 104, 105, 106, 107],
    'Gender': ['Male', 'Female', np.nan, 'Female', 'Male', np.nan, 'Female'],
    'City': ['New York', 'Los Angeles', 'Chicago', np.nan, 'New York', 'Chicago', 'New York']
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
```

Mode Imputation

- Step 3: Check for Missing Values

```
print("\nMissing Values Count:")  
print(df.isnull().sum())
```

- Output:

Missing Values Count:

Customer_ID 0

Gender 2

City 1

dtype: int64

- Here, we observe that:
 - "Gender" has 2 missing values.
 - "City" has 1 missing value.

Mode Imputation

- Step 4: Apply Mode Imputation

```
# Filling missing values with the mode of the respective column
df['Gender'].fillna(df['Gender'].mode()[0], inplace=True)
df['City'].fillna(df['City'].mode()[0], inplace=True)
print("\nDataFrame After Mode Imputation:")
print(df)
```

- Step 5: Verify Missing Values Again

```
print("\nMissing Values Count After Imputation:")
print(df.isnull().sum())
```

Mode Imputation

Explanation of the Code

- We created a sample dataset where some values in the "Gender" and "City" columns are missing.
- We used `df.isnull().sum()` to check how many missing values exist.
- We applied mode imputation using `df[column].mode()[0]` to replace missing values with the most frequent value.
- Finally, we confirmed that no missing values remain after imputation.

Mode Imputation

When to Use Mode Imputation?

- When dealing with categorical data (e.g., Gender, City, Occupation).
- When missing data is relatively small (not more than 10-15%).
- When we want to avoid introducing bias that could come from random or mean-based imputation.

K-Nearest Neighbors Imputation

- Data scientists determine a data point's nearest neighbors and approximate an estimate based on the values of points closest and other variables.
- The data scientist must select the number of nearest neighbors and the distance metric.
- KNN can identify the most frequent value among the neighbors and the mean among the nearest neighbors.

How KNN Imputation Works:

- Identify Missing Values:
 - Locate the columns and rows with missing values in the dataset.
- Normalize/Scale the Data:
 - KNN is distance-based, so it's important to scale or normalize the data to ensure all features contribute equally to the distance calculation.
- Fit KNN on the Dataset:
 - Use the rows without missing values to train a KNN model.
- Predict Missing Values:
 - For each row with missing values, find the k nearest neighbors (rows) in the dataset that have no missing values for the same feature.
 - Use the values of these neighbors to predict the missing value (e.g., by taking the mean for continuous variables or the mode for categorical variables).
- Fill the Missing Values:
- Replace the missing values with the predicted values.

K-Nearest Neighbors Imputation

```
import pandas as pd
import numpy as np
from sklearn.impute import KNNImputer
from sklearn.preprocessing import MinMaxScaler

# Sample dataset with missing values
data = {
    'Age': [25, 30, np.nan, 35, 40, np.nan, 28],
    'Salary': [50000, 60000, 55000, np.nan, 70000, 65000, np.nan],
    'Experience': [1, 3, 2, 5, np.nan, 4, 2]
}

# Create a DataFrame
df = pd.DataFrame(data)

# Display the original dataset
print("Original Dataset:")
print(df)

# Step 1: Normalize/Scale the data (KNN is distance-based)
scaler = MinMaxScaler()
df_scaled = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)

# Step 2: Use KNNImputer to fill missing values
imputer = KNNImputer(n_neighbors=2) # Use 2 nearest neighbors
df_imputed = pd.DataFrame(imputer.fit_transform(df_scaled), columns=df.columns)

# Step 3: Inverse transform to get back the original scale
df_imputed = pd.DataFrame(scaler.inverse_transform(df_imputed), columns=df.columns)

# Display the dataset after imputation
print("\nDataset after KNN Imputation:")
print(df_imputed)
```

Dealing with Outliers in Data Analytics

What is Outliers

- Outliers are data points that significantly differ from other observations in a dataset.
- It can be an extremely high or low value compared to the other observations in a dataset.
- They can arise due to errors, variability in data, or genuine extreme values.
- Identifying and handling outliers is crucial because they can distort statistical analyses and machine learning models.

Types of Outliers

Types of Outliers

- Global (Point) Outliers: A single data point that deviates significantly from the rest of the dataset.
 - Example: A person's age recorded as 150 in a human population dataset.
- Contextual Outliers: A value that is an outlier only in a specific context.
 - Example: A temperature of 30°C is normal in summer but an outlier in winter.
- Collective Outliers: A group of data points that, together, deviate from the expected pattern.
 - Example: A sudden spike in website traffic due to a cyberattack.

Outliers Effect

Why Are Outliers Problematic?

- Affect Statistical Analysis: Outliers can distort mean, standard deviation, and correlation.
- Impact Machine Learning Models: ML models may overfit to extreme values, reducing accuracy.
- Mislead Decision Making: Incorrect data can result in poor business decisions.

Outlier Detection Using Isolation Forest

- Isolation Forest is an unsupervised machine learning algorithm used for outlier detection.

Key Concepts of Isolation Forest:

- Isolation:
 - Outliers are isolated by randomly selecting features and splitting values.
 - The number of splits required to isolate a data point is used as a measure of its abnormality.
- Anomaly Score:
 - The anomaly score is calculated based on the average path length (number of splits) required to isolate a data point.
 - A higher anomaly score indicates a higher likelihood of being an outlier.
- Threshold:
 - A threshold is set to classify data points as outliers or inliers (normal points).

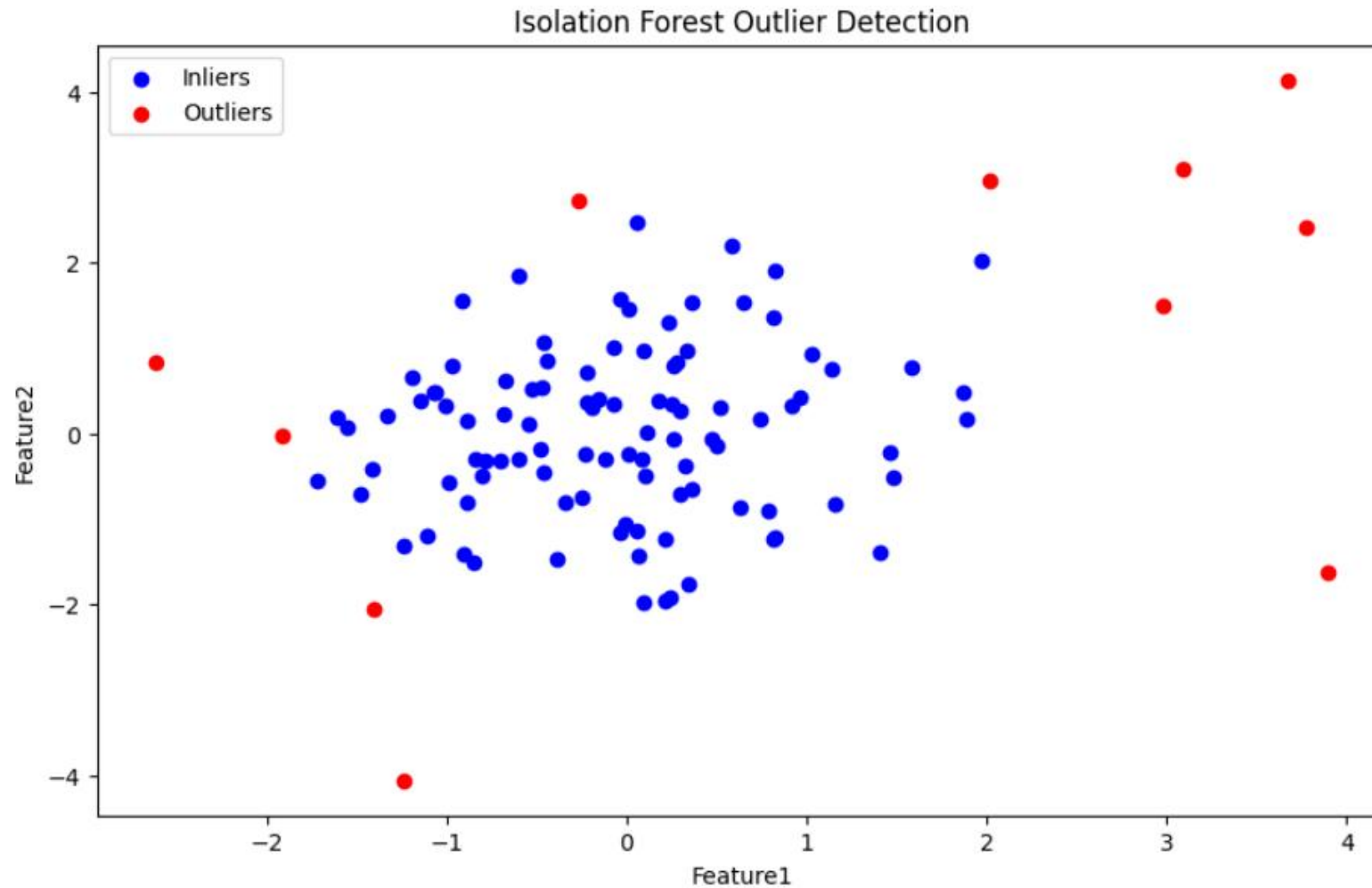
Code

- Exist in a separated file

Code Explanation

- Dataset Creation:
 - A synthetic dataset is created with 100 normal data points and 10 outliers.
 - Normal data points are generated using a Gaussian distribution (`np.random.normal`).
 - Outliers are generated using a uniform distribution (`np.random.uniform`).
- Standardization:
 - The data is standardized using `StandardScaler` to ensure all features are on the same scale. This is important because Isolation Forest is sensitive to feature scales.
- Isolation Forest Model:
 - The `IsolationForest` model is initialized with `contamination=0.1`, which means the algorithm expects 10% of the data to be outliers.
 - The model is trained on the scaled data.
- Outlier Prediction:
 - The `predict` method returns -1 for outliers and 1 for inliers.
- Visualization:
 - The results are visualized using a scatter plot. Inliers are shown in blue, and outliers are shown in red.
- Output:
 - The detected outliers are printed in a tabular format.

Code Results



Adv and DisAdv of Isolation Forest

- Efficient for high-dimensional datasets.
 - Works well with both numerical and categorical data (with proper encoding).
 - Does not require labeled data (unsupervised).
- Disadvantages of Isolation Forest:
 - May struggle with datasets where the normal data and outliers are not well-separated.
 - Requires tuning of the contamination parameter.
- When to Use Isolation Forest:
 - When you suspect the dataset contains outliers.
 - When the dataset is large and high-dimensional.

Handling Outlier

- Methods to Handle Outliers

1. Removing Outliers:

- Suitable when the outliers are due to data entry errors.

2. Transforming Data (Normalization & Log Transformation)

- Log Transformation: Reduces the effect of large values.

```
df['Log_Values'] = np.log1p(df['Values'])
```

Outliers

When to Keep Outliers?

- When they contain valuable information
 - Example: A financial dataset where fraud detection depends on unusual transactions.
- When they naturally occur
 - Example: Athlete records in sports (Usain Bolt's 100m sprint time).
- When the dataset is small
- Removing outliers may lead to a loss of essential information.